Integer Factorization as an Ising problem

Advith Desu

IIT Madras

May 5, 2026

GPU Kernel

Digital Annealer 1 (DA1) manually launches kernels by assigning specific threads to do the energy calculations. See GitHub Link for the code used.

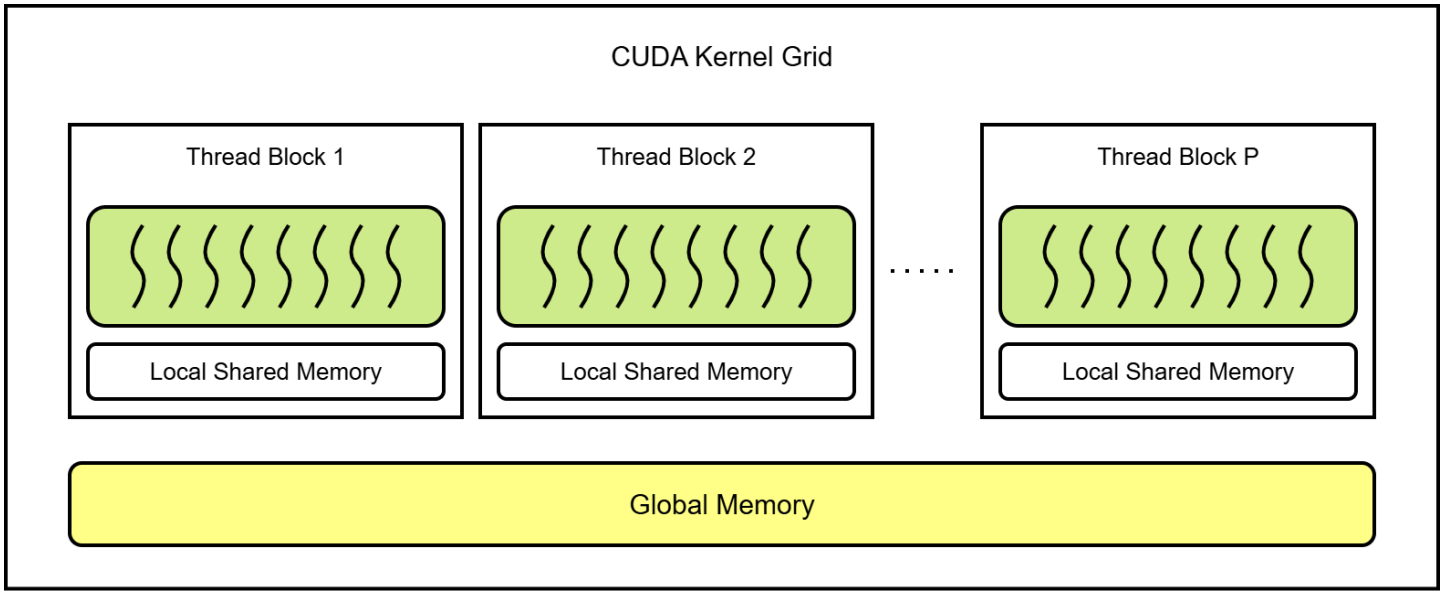


Figure: GPU Threads and Thread blocks.

GPU Thread Calculation in DA1

Each thread i calculates the local field $L_i = \sum_j J_{ij} s_j + h_i$ and the energy difference is then calculated as $-2 \times s_i \times L_i$ depending on which i^{th} spin flipped.

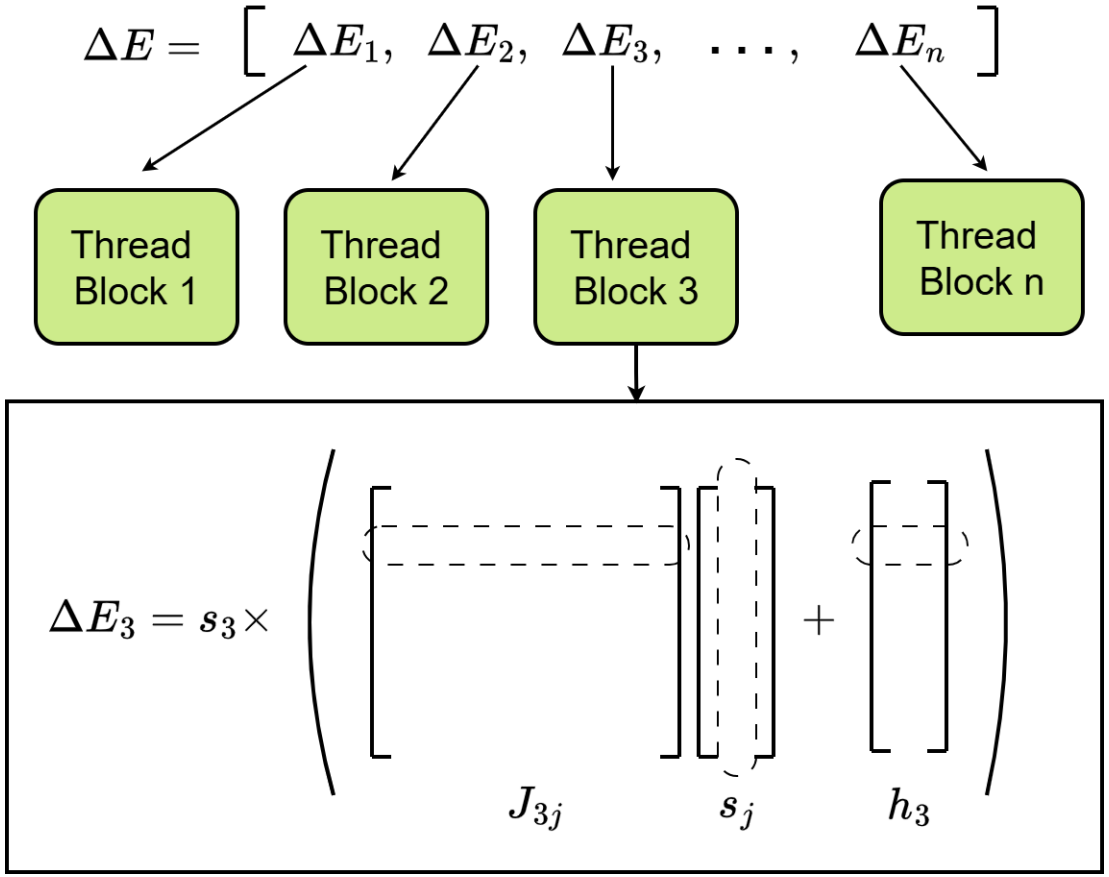


Figure: Energy calculation in DA1.

Pseudo-code for Digital Annealer 1 (DA1)

Algorithm Pseudo-code for Digital Annealer 1

Require: Initial temperature, number of iterations *iters*, connectivity matrix $J[N][N]$ and magnetic field $h[N]$

Ensure: Spin configuration $s[N]$

```
Initialization: Randomly initialize spins s[N], k = 0
Copy J[N][N] and h[N] to GPU memory
while k ≤ iters do
    for i = 1 : N do                                  ▷ N blocks for N-spin model
        for j = 1 : N/P do
            Calculate partial sum of ΔH(s_i) by computing J[i][j] · s[j]
        end for
        for p = 1 : log(P) do                         ▷ Using P threads
            Reduction of partial sum of ΔH(s_i)
        end for
        Calculate P_B(ΔH, T) for ΔH(s_i) to determine state of spin s_i
    end for                                           ▷ For all N spins in ensemble
    Update all possible spins
end while
Decrease temperature T linearly
```

Pseudo-code for DA1 with fractional flips I

Require: Initial temperature, number of iterations *iters*, connectivity matrix $J[N][N]$, magnetic field $h[N]$, flip fraction ρ

Ensure: Spin configuration $s[N]$

```
Initialization: Randomly initialize spins s[N]
Copy J, h, and s to GPU memory
while k ≤ iters do
    A ← ∅                                          ▷ List of Metropolis-accepted spins
    for i = 1 : N in parallel do                   ▷ N GPU blocks
        for j = 1 : N/P do                         ▷ P threads per block
            Compute partial sum of local field contribution:
```

$$S_i \mathrel{+}= J[i][j] \cdot s[j]$$

```
        end for
        for p = 1 : log(P) do                      ▷ Parallel reduction
            Reduce partial sums to obtain full local field S_i
        end for
        Compute energy change:
```

$$\Delta H_i = -2\, s[i]\, (S_i + h[i])$$

Pseudo-code for DA1 with fractional flips II

```
        Compute Metropolis probability:
```

$$P_B = \min\{1, \exp(-\Delta H_i / T)\}$$

```
        if random() < P_B then
            Append index i to A
        end if
    end for                                  ▷ None of the spins are flipped
    Let M ← |A|                              ▷ Total number accepted
    k ← ⌈ρ · M⌉                              ▷ Exact number of flips to perform
    Randomly shuffle the list A (Fisher–Yates)
    A_k ← first k elements of A              ▷ Random subset of size k
    for each i ∈ A_k in parallel do
        Flip spin: s[i] ← −s[i]
    end for
    Update iteration counter: k ← k + 1
end while
Decrease temperature T linearly
return s[N]
```

Factorization Problem

- Given a natural number N, find P, Q such that

$$N = PQ$$

- Hard inverse problem underlying RSA security.
- Idea: map factorization $\rightarrow$ optimization problem.
- Solve using annealing-based techniques.

Binary Representation and Assumptions

Assumptions:

- N is odd.
- Factors have equal bit-length

$$n = \left\lceil \tfrac{1}{2} \log_2 N \right\rceil .$$

Binary form:

$$P = \sum_{k=0}^{n-1} 2^k p_k, \quad Q = \sum_{k=0}^{n-1} 2^k q_k, \quad p_k, q_k \in \{0, 1\}.$$

Fixed bits:

$$p_0 = q_0 = 1, \qquad p_{n-1} = q_{n-1} = 1.$$

Column-wise Multiplication

- Construct binary long multiplication table.
- Introduce carry variables $s_{i,j}$.
- Each column gives a constraint:

$$C_i = N_i - \sum_j q_j p_{i-j} - \sum s_{j,i} + \sum 2^j s_{i,i+j}.$$

- Correct factors satisfy $C_i = 0$.

Energy Function

Optimization objective:

$$\min_{\{p_i, q_i, s_{ij}\}} \sum_i (C_i)^2.$$

- Binary variables: factor bits + carry bits.
- Ground state corresponds to valid factorization.
- Alternative direct method:

$$\left(N - \sum p_i 2^i \sum q_j 2^j\right)^2$$

  but gives fewer preprocessing opportunities.

Core Reduction Rules

For binary variables $x_i \in \{0, 1\}$:

Sum rules

$$\sum_i x_i = n \Rightarrow x_i = 1, \qquad \sum_i x_i = 0 \Rightarrow x_i = 0.$$

Complement rule

$$x_1 + x_2 = 1 \Rightarrow x_2 = 1 - x_1.$$

Power rule

$$x^m = x, \quad m \geq 2.$$

Coefficient and Parity Rules

General linear constraint:

$$\sum_i c_i x_i + C = 0.$$

Let

$$S_+ = \sum_{c_i > 0} c_i, \quad S_- = \sum_{c_j < 0} |c_j|.$$

- Large coefficients can force $x_i = 0$ or 1.
- Parity constraints:

$$x_1 + x_2 + \text{even} = \text{even} \Rightarrow x_1 = x_2,$$
$$x_1 + x_2 + \text{even} = \text{odd} \Rightarrow x_1 + x_2 = 1.$$

Example: $N = 15$ Reduction

Initial clauses (example):

$$C_1 : -p_1 - q_1 + 2s_{1,2} + 1 = 0,$$
$$C_2 : -p_1 q_1 - p_2 - q_2 - s_{1,2} + 2s_{2,3} + 4s_{2,4} + 1 = 0.$$

Applying rules:

- Fix carries $s_{1,2} = s_{2,4} = 0$.
- Complement relation: $q_1 = 1 - p_1$.
- Power rule: $p_1^2 = p_1$.

Final reduced constraint:

$$2p_1 p_2 - p_1 - p_2 + 1 = 0 \Rightarrow p_1 + p_2 = 1.$$

J matrix Visualization I

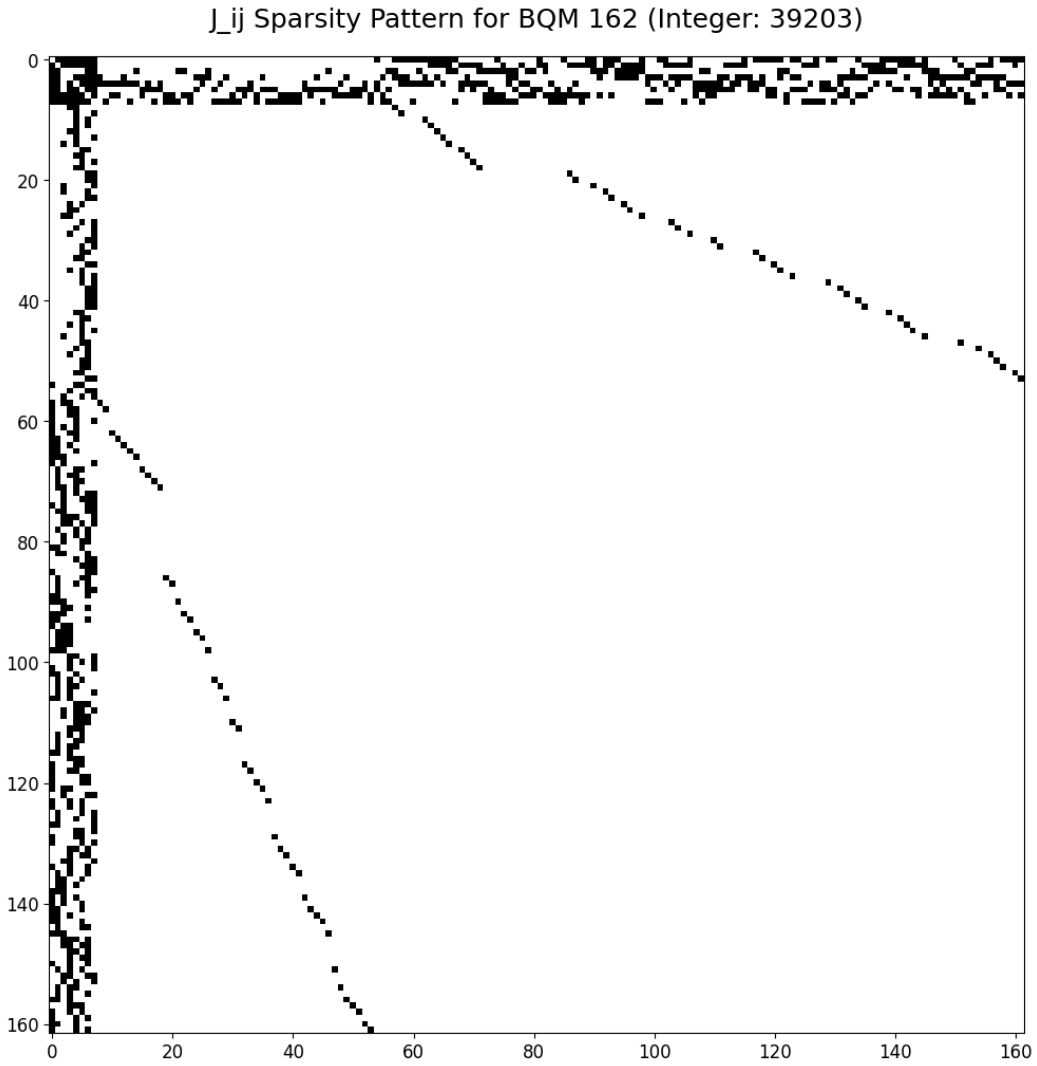


Figure: Sparsity of J matrix for 162 BQM (Integer 39203).

Sparse Coupling Matrix Representation (CSR)

Problem: The Ising coupling matrix J_{ij} is very large but mostly zero. Storing it as a dense $N \times N$ matrix wastes memory and bandwidth.

Key idea: Store *only the nonzero couplings* using Compressed Sparse Row (CSR) format. CSR represents a sparse matrix using three arrays:

$$\texttt{row_ptr}[i] \quad \text{start index of row } i$$

$$\texttt{col_idx}[k] \quad \text{column index of nonzero entry}$$

$$\texttt{values}[k] \quad \text{value of } J_{ij}$$

Only nonzero entries are stored.

Memory cost: $\mathcal{O}(N + \text{nnz})$ instead of $\mathcal{O}(N^2)$

Time Taken for J construction and CSR DA1

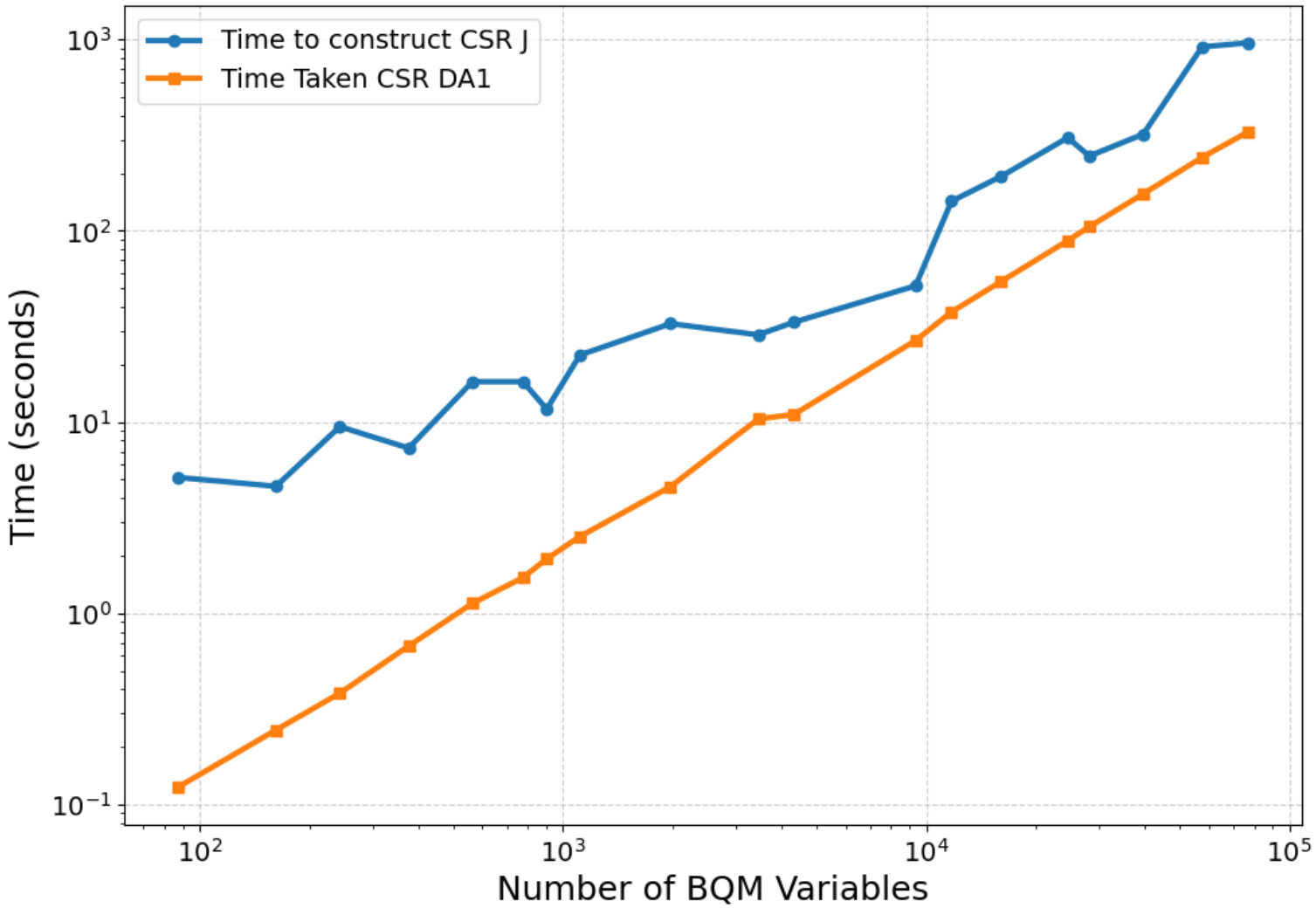


Figure: BQM variables vs time taken for J construction and DA1.

Number of Densely connected spins with scale

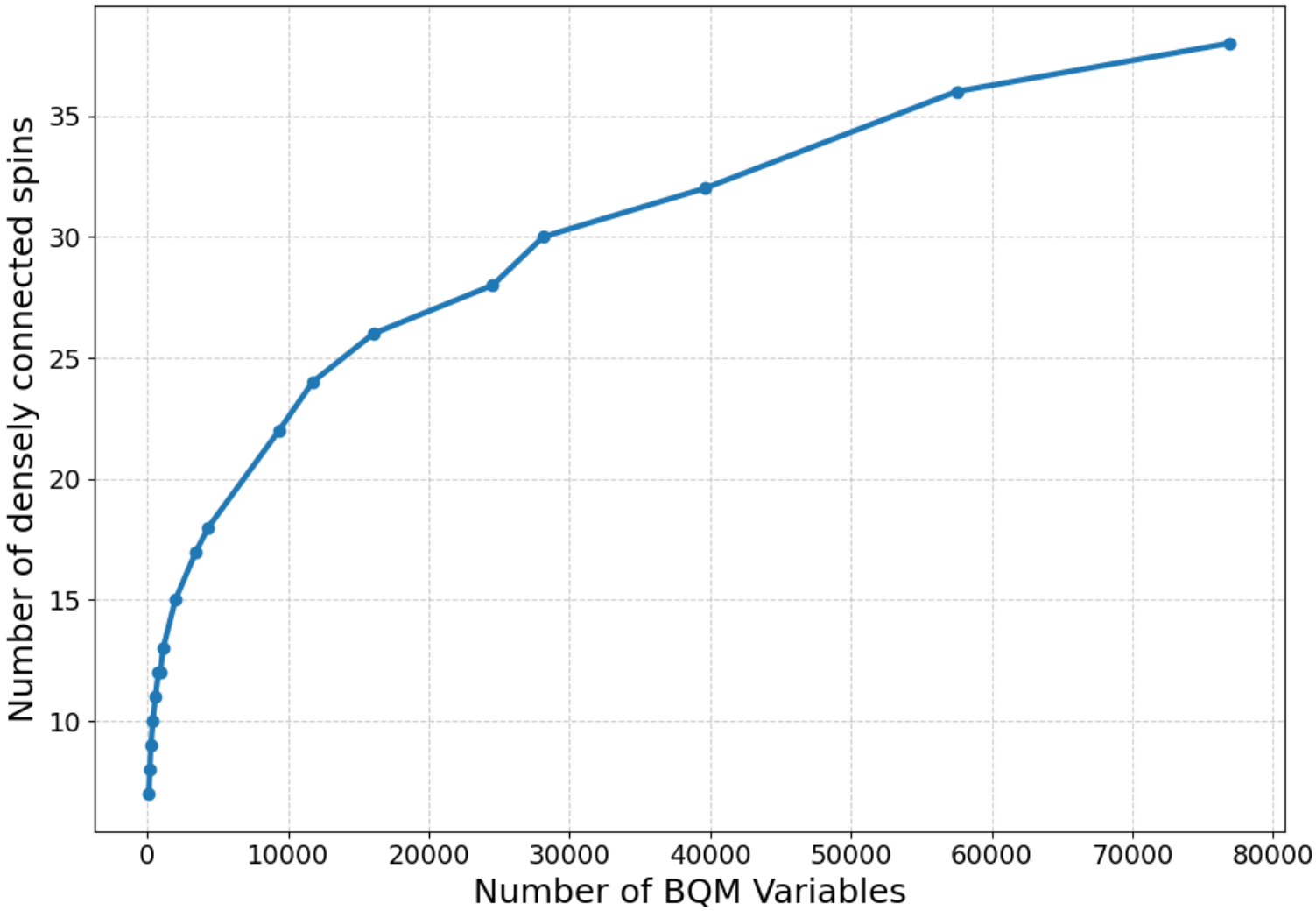


Figure: BQM variables vs number of densely connected spins.

Number of Non-zero values in J matrix with scale

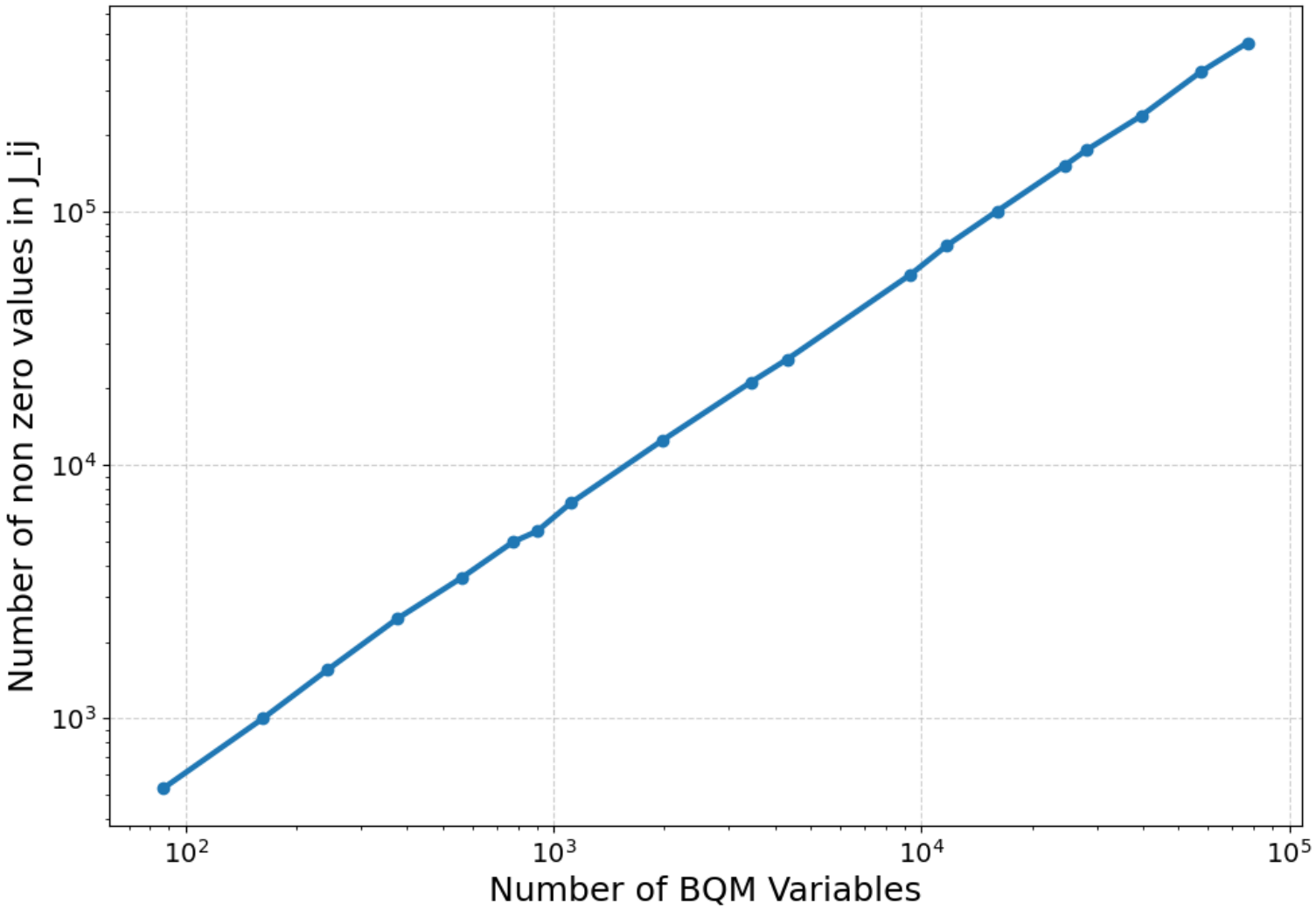


Figure: BQM variables vs number of non zero values in J_{ij}.

Energy and Average Energy difference with scale

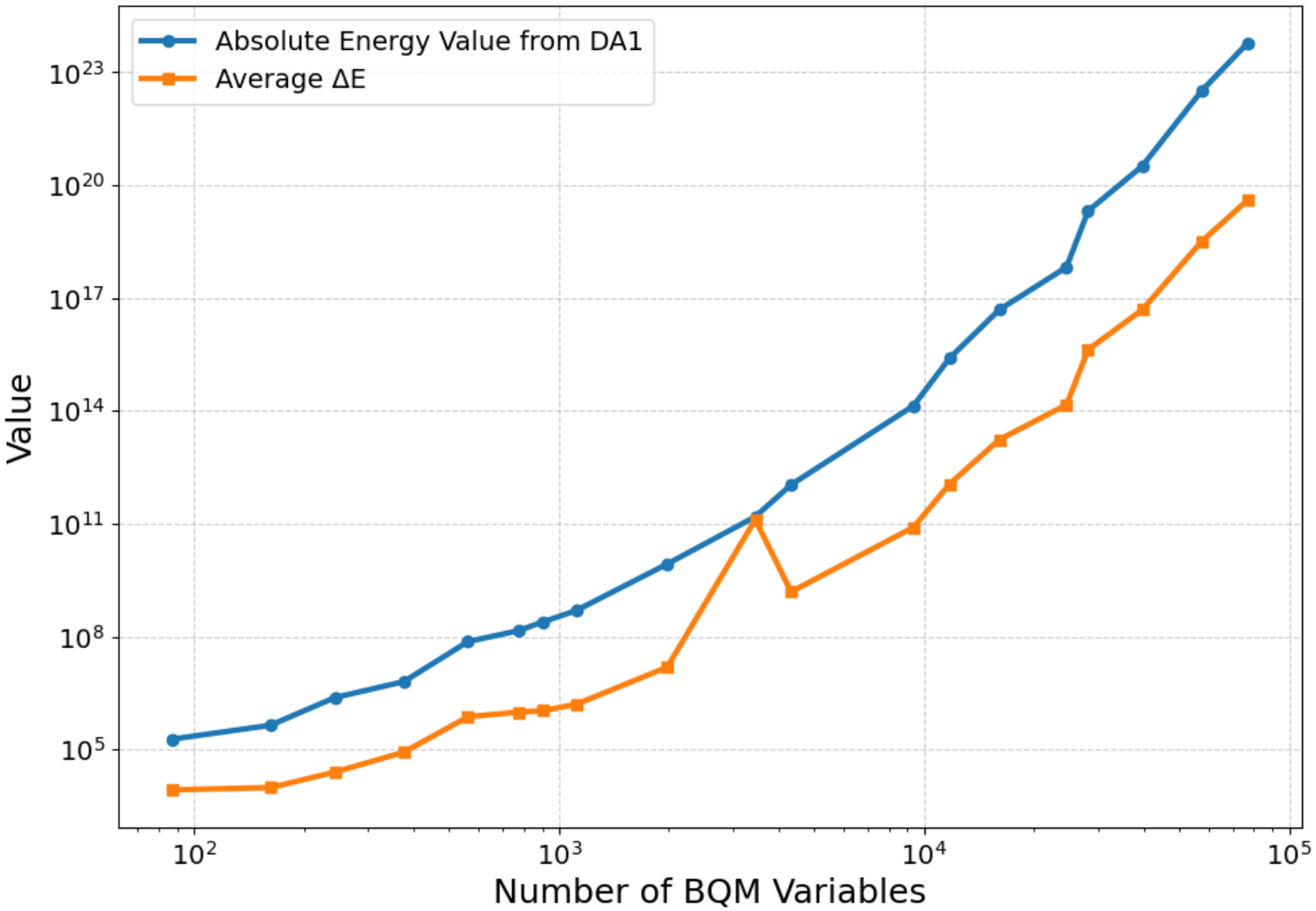


Figure: BQM variables vs absolute energy value and average δE values.

Trend-lines

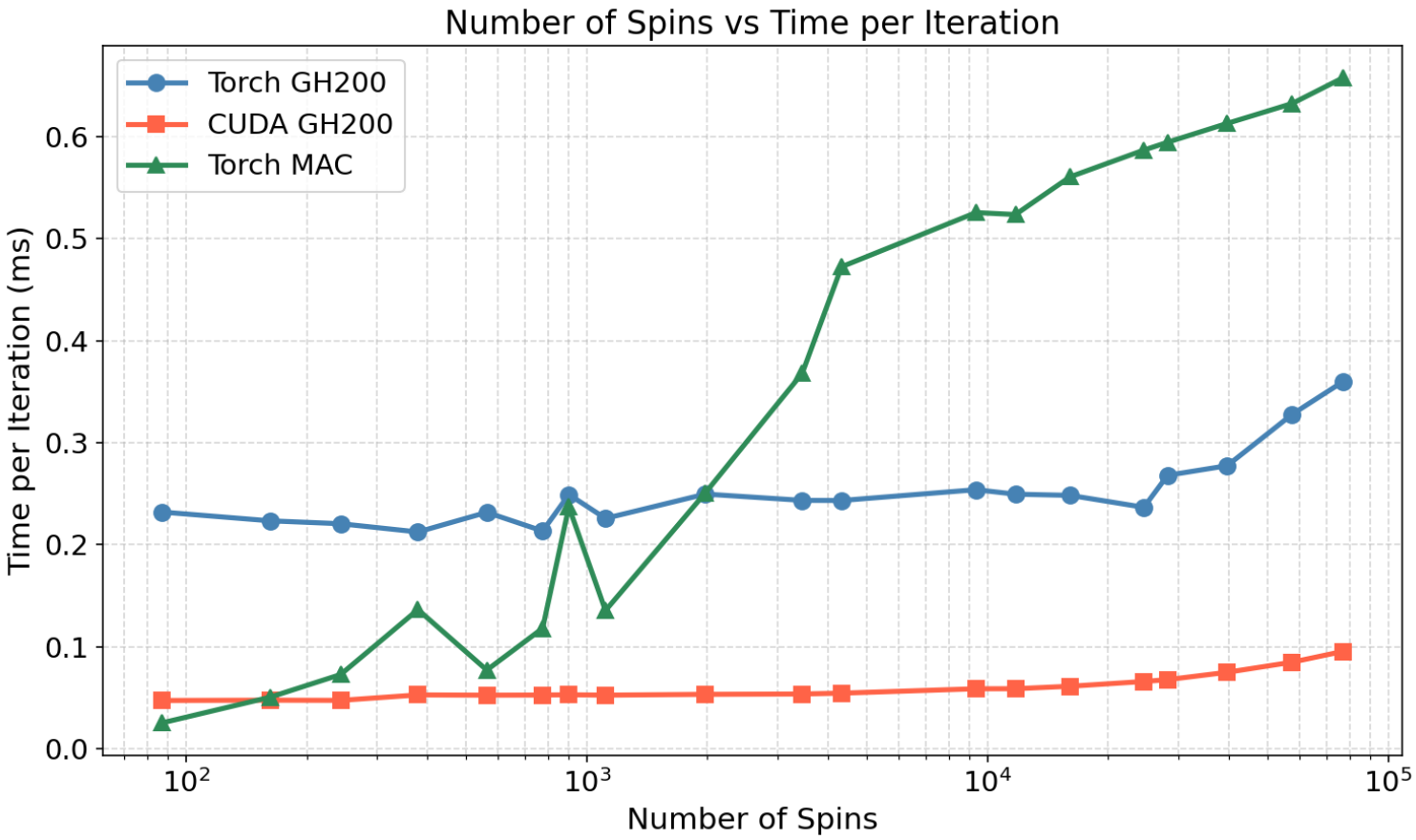
$$\text{Number of BQM variables (bqm)} \leq (\lceil log_2 N \rceil)^3$$

$$\text{Number of non-zero } J_{ij} \text{ elements (nnz)} = 6.08 \times \text{bqm} + 680$$

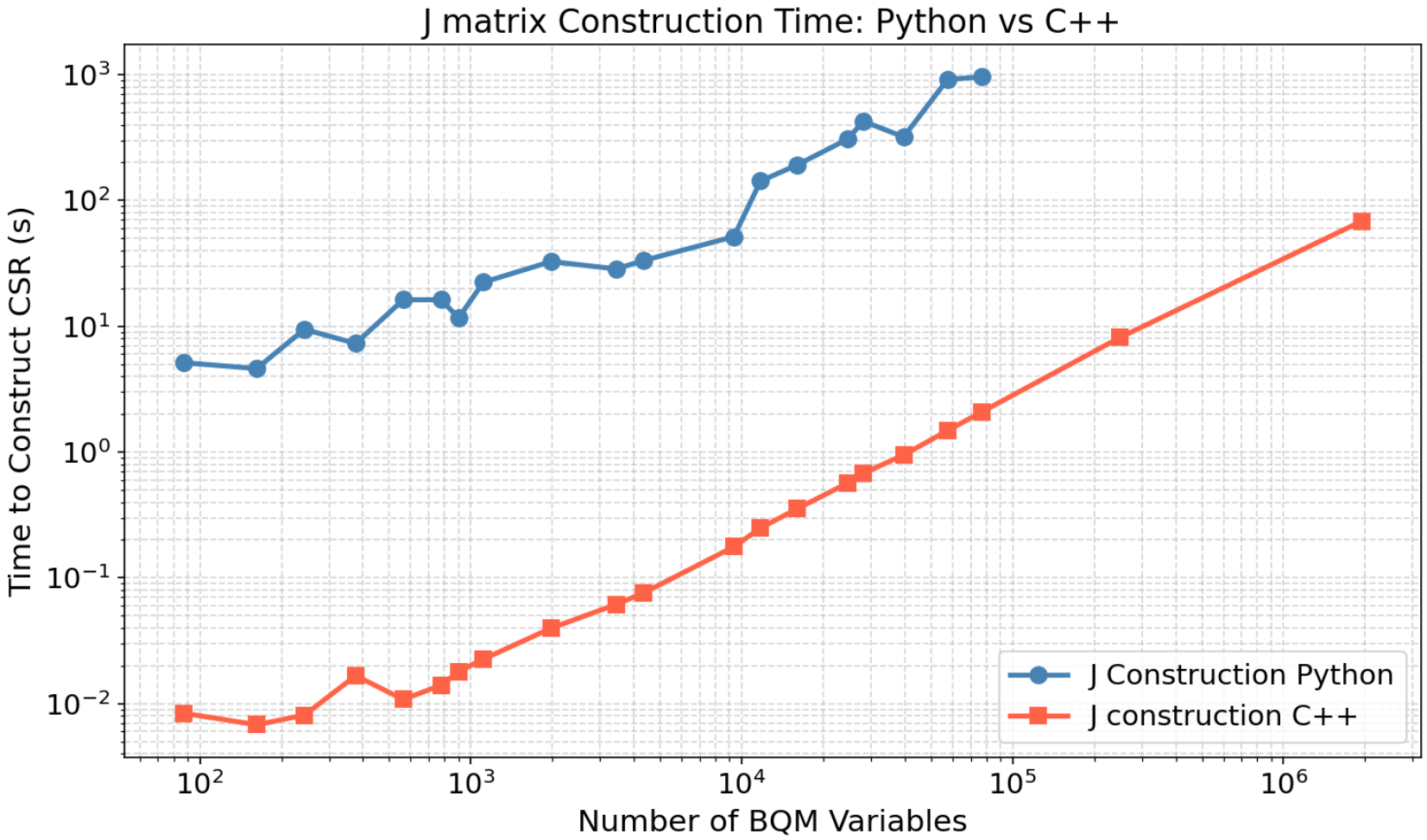
$$\text{time}(s) = 4.22 \times 10^{-3} \times \text{bqm}$$

- For a 128 bit number (N), it would have approximately 2 million bqm variables, 194 MB to store J_{ij} and would take 2.5 hrs to run on DA1.
- For a 256 bit number (N), it would have approximately 17 million bqm variables, 1.5 GB to store J_{ij} and would take 19.66 hrs to run on DA1

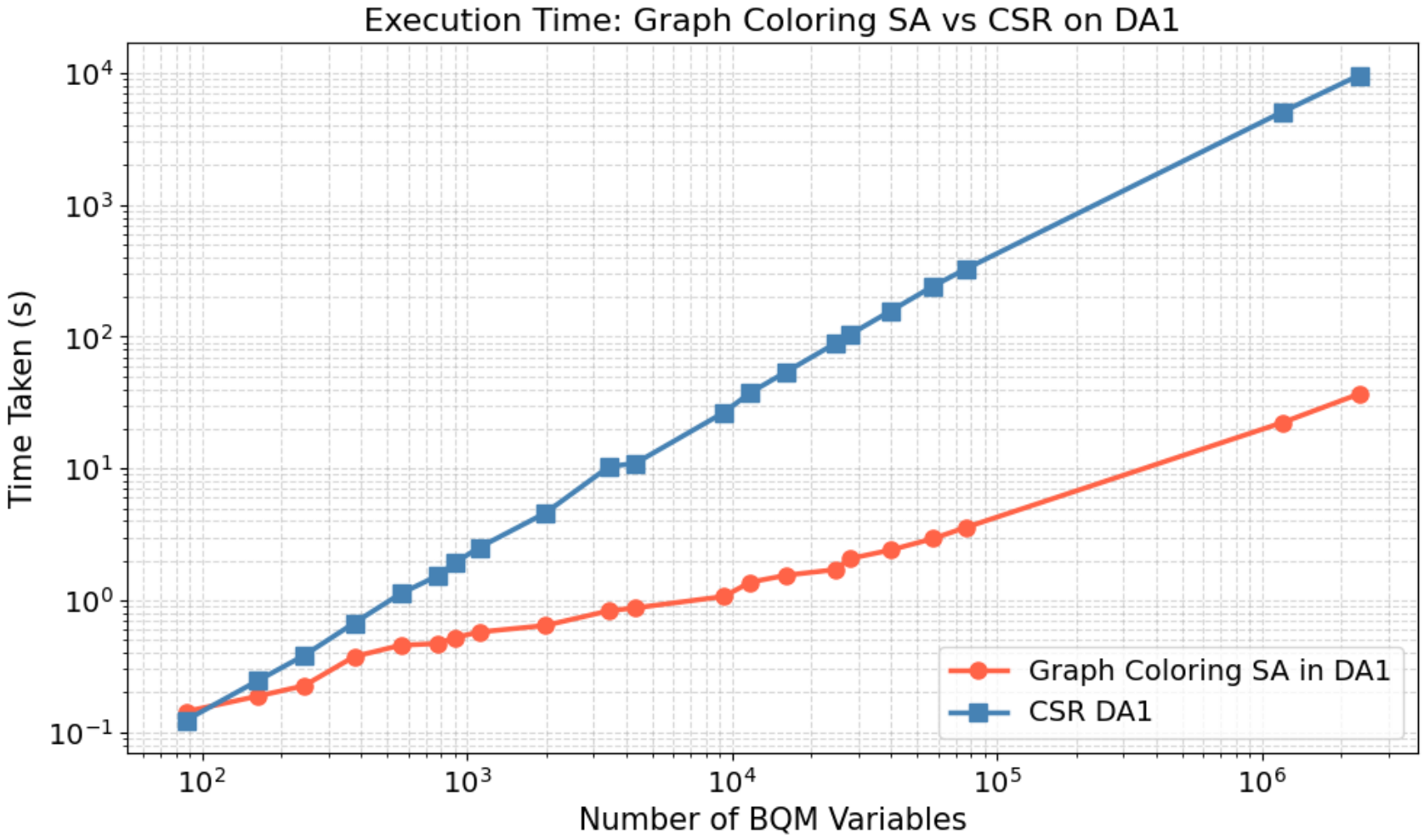
Annealing Time per Iteration vs Number of Spins



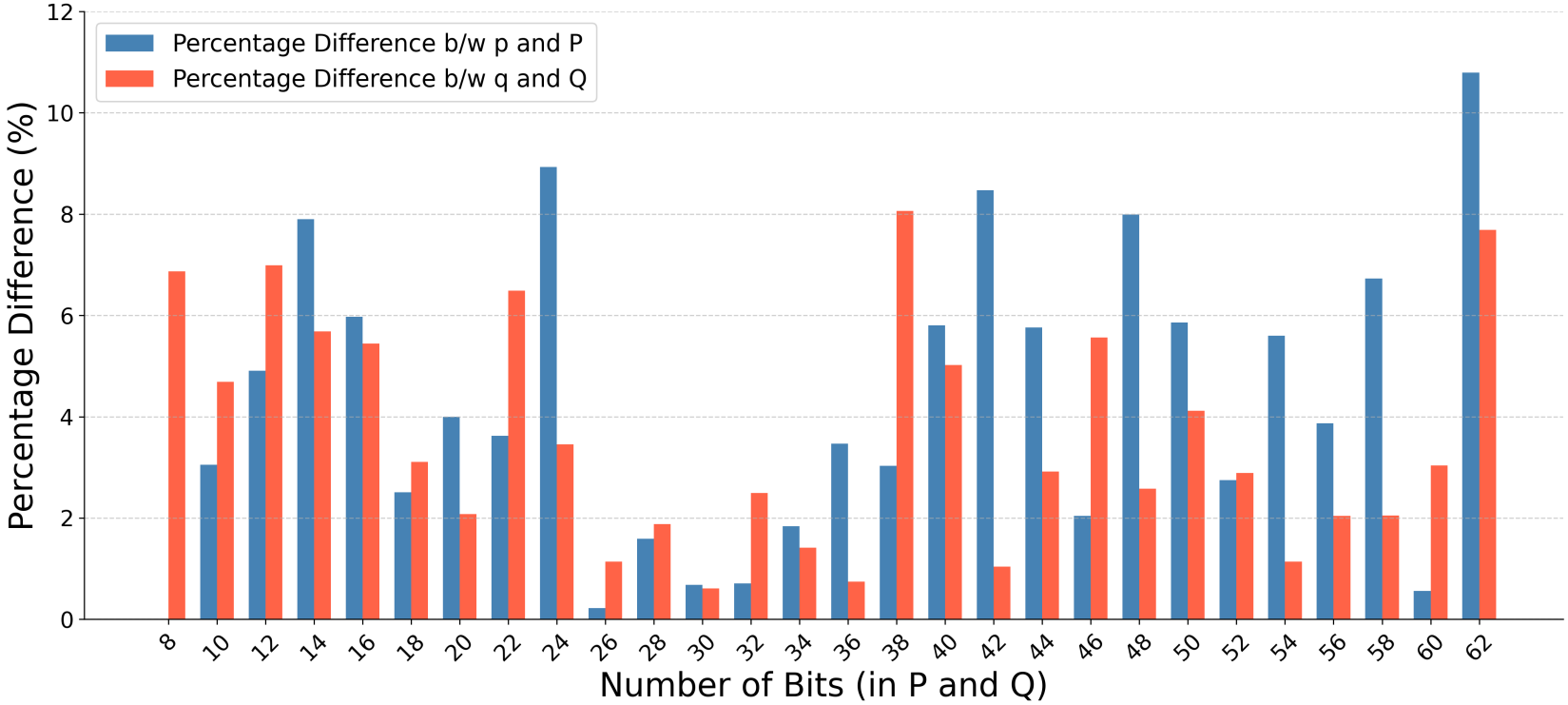
Construction time for CSR J



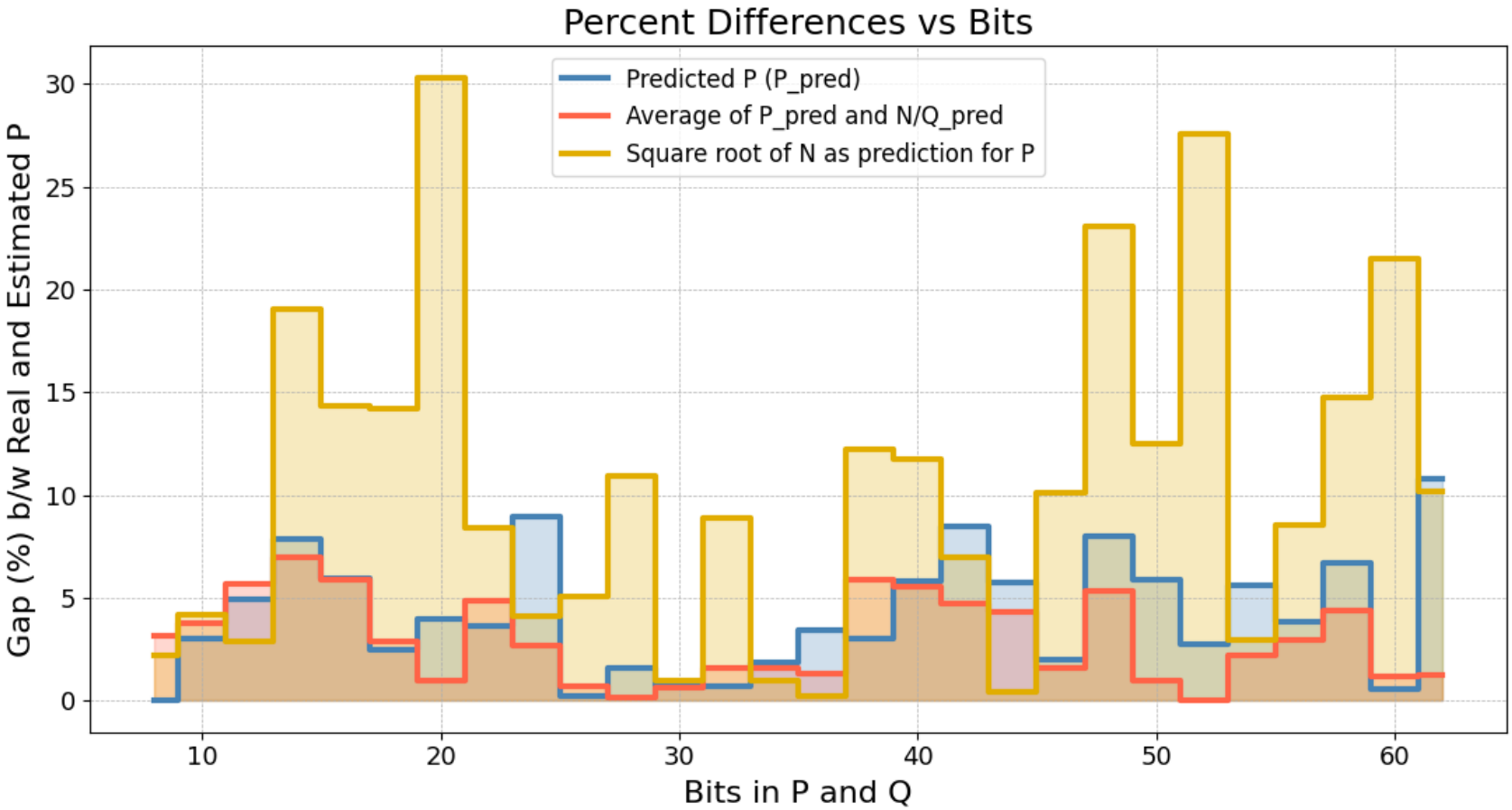
Execution Time Comparison



Solution Quality from Graph-Colored Simulated Annealing

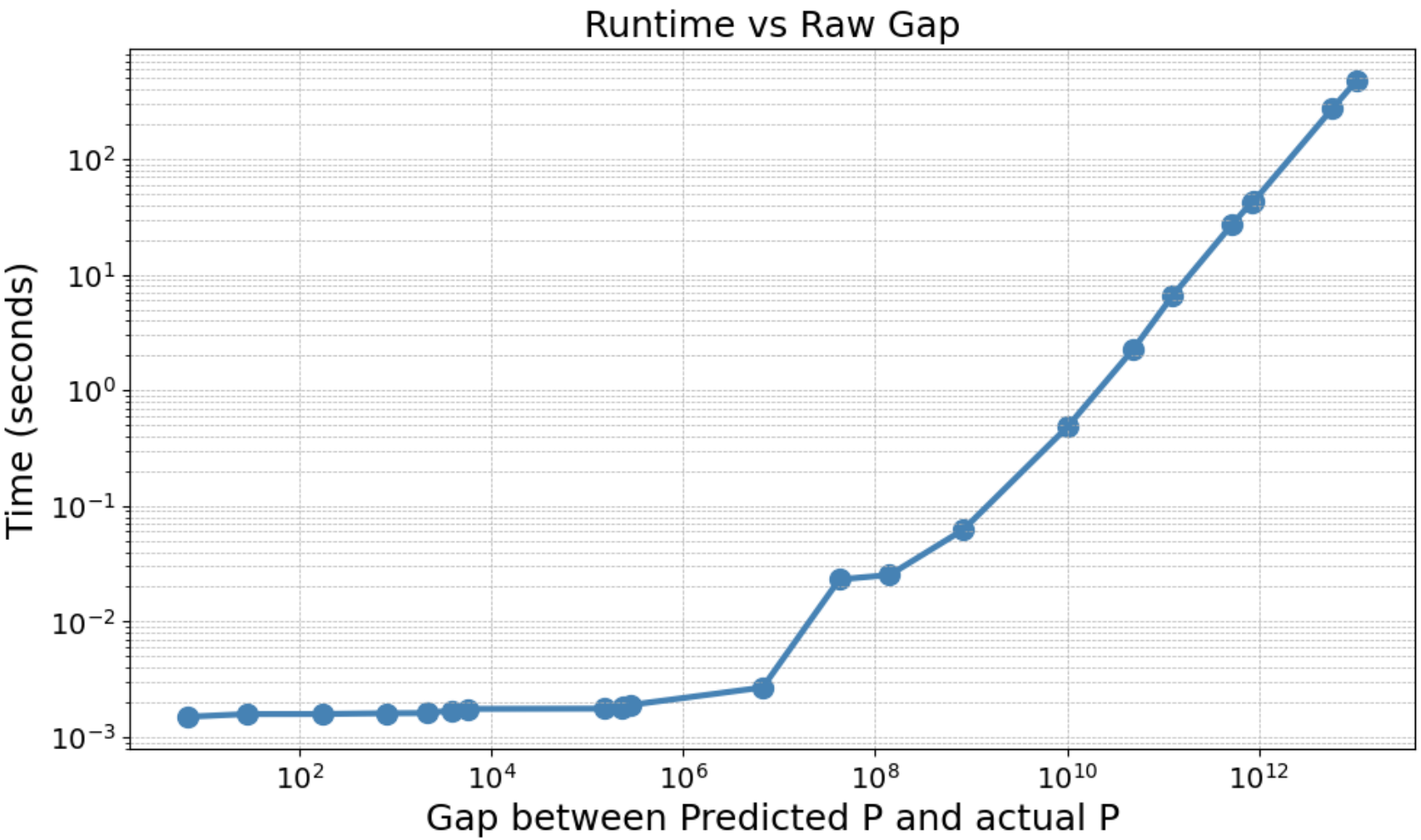


Solution Quality Measured by Gap between Real and Estimated P



1. Average (%) difference with square root N as the estimate = **10.33%**
2. Average (%) difference with P_{pred} from Simulated Annealing = **4.23%**
3. Average (%) difference with mean of P_{pred} and N/Q_{pred} = **2.98%**

Time Taken to Find Exact primes by Brute Force



$$\text{Time Taken (s)} = \frac{3.32 \times 10^{-9}}{\text{Threads}} \times \text{Gap (\%)} \times 2^{n}$$